RESTAURANT GROWTH OS

Junior Developer Technical Handbook

Version 1.0 | 27 October 2026 | Working baseline

A practical implementation guide for junior developers. The project knowledge base and approved decision log remain authoritative.

Version 1.0 | 27 October 2026 | Working baseline

This is an implementation guide, not approval for unresolved decisions. The project knowledge base and approved decision log remain authoritative. If something is marked OPEN, ask rather than guess.

1. How to use this handbook

Before starting a task

1. Read `AGENT_RULES.md`, the assigned task, and the linked specification.

2. Check task dependencies and inspect existing code before editing.

3. State the expected behavior, acceptance criteria, risks, and unknowns.

4. Ask questions when a requirement is ambiguous; do not invent product behavior.

5. Implement the smallest complete change and add tests.

6. Run relevant lint, typecheck, unit/integration tests, and build checks.

7. Review the diff, remove debug code, check for secrets, and submit a focused PR.

8. Update documentation or decision records if behavior or approved decisions change.

Decision labels

- **Approved principle:** do not change without owner approval.
- **Proposed:** validate with a proof of concept and get approval.
- **Open:** not decided; do not implement assumptions as facts.
- **Example:** illustrative only.

2. Product and business context

Restaurant Growth OS helps owners understand performance, find weaknesses, improve sales/profitability, and measure whether actions worked. POS, menu, orders, kitchen, inventory, purchasing, expenses, payments and staff workflows provide operational data; the product's differentiator is an evidence-based action loop.

Plans

- **Starter:** intended for one outlet.
- **Growth:** includes three outlets; extra outlet pricing is to be approved.
- **Business:** starts with a trial request and platform-admin approval.
- Exact feature matrices, prices, currency, trial duration, add-ons, taxes and downgrade-over-limit behavior are **OPEN**.

Data truth

Missing data is not zero. Recipe-based ingredient consumption is theoretical, not actual counted consumption. Label metrics measured, estimated or incomplete. Do not claim net profit if material expense categories are missing.

Two payment domains

1. **Platform billing:** our SaaS charges to restaurant businesses.

2. **Diner payments:** payments restaurant customers make to restaurants.

Keep the models, permissions, webhooks, reconciliation and audit trails separate.

3. Proposed architecture

Proposed technology baseline

- Mobile: React Native + TypeScript.
- Web: React/Next.js + TypeScript.
- Backend: NestJS + TypeScript.
- Transactional database: PostgreSQL.
- API: GraphQL queries, mutations and subscriptions where appropriate; REST for selected webhooks, callbacks and file workflows.
- Deployment: Dockerized API and worker processes, Terraform-managed infrastructure.
- Redis, queues and other services should be introduced only when approved and justified by reliability or measured need.

Start with a modular monolith

Keep domain modules in one backend codebase initially, but allow API and worker processes to run and scale separately. Extract microservices only when measured scaling, reliability, ownership or deployment needs justify their operational cost. Do not create a distributed system just because the product may grow.

Logical flow

Mobile / Web

|

HTTPS: GraphQL + selected REST

v

NestJS API

- Authentication and tenant/outlet authorization

- Domain services: menu, orders, kitchen, inventory, reporting, billing

- PostgreSQL transactions

|

+---- PostgreSQL (source of truth)

|

+---- Transactional outbox / queue

|

v

Worker process

- reports/imports

- email/Teams notifications

- external integrations

GraphQL subscription delivery -> authorized live KDS/POS/owner clients

Request lifecycle

Authenticate the caller; derive tenant membership on the server; authorize outlet and action; validate input; call domain services; commit the transaction; record audit data; publish or enqueue events using the approved reliable pattern; return a typed response and correlation ID.

Horizontal scaling

Application instances should be stateless. Durable business/session/job state must not live only in local memory or container disk. Multi-instance subscriptions and workers need shared delivery/state. “10,000 tenants” is not a workload definition: establish outlets, concurrent users, orders/second, peak multipliers, reports, subscription connections, retention, availability and recovery targets, then load-test.

4. Repository structure and module boundaries

Suggested layout:

apps/api/ NestJS API entrypoint

apps/worker/ background job entrypoint

apps/web/ Next.js web app

apps/mobile/ React Native app

packages/domain/ pure business rules and value objects

packages/database/ schema, migrations, repositories

packages/contracts/DTOs and event contracts

docs/ architecture, decisions and runbooks

Each domain module owns its rules and exposes a small service interface. Do not reach into another module's private repository. GraphQL resolvers and REST controllers should call domain services; they should not duplicate business logic. Use stable error codes and safe messages. Never return stack traces or SQL errors to clients.

Definition of done: acceptance criteria pass; relevant tests are added; lint/typecheck/build pass; tenant/security impact is reviewed; migrations are tested; behavior changes are documented; PR evidence is provided.

5. Database and multi-tenancy

Core entities

Likely entities include CustomerAccount, Tenant/RestaurantOrganization, optional Brand/FranchiseGroup, Outlet, User, Membership, Role/Permission, Menu, MenuItem, Order, OrderItem, Payment, StockMovement, Supplier,

Purchase, Expense, Subscription, PlanVersion, Entitlement, Invoice, PaymentAttempt, AuditEvent and NotificationDelivery. Final relationships depend on approved decisions.

Franchise model

Three outlets owned by one restaurant business are normally one tenant with three outlets. Independently owned franchisees may require separate tenants linked to a brand group. Payer, data visibility, consolidated reporting, and staff memberships must be decided before finalizing schema.

Tenant-scoped records

Business rows should carry `tenant_id` and, where relevant, `outlet_id`, or have an equivalent foreign-key path. Use tenant-aware uniqueness constraints. Add indexes based on measured query patterns, not guesses.

Never trust client IDs

A client-supplied `tenantId` or `outletId` is a request, not proof of access. Resolve membership from the authenticated identity and authorize each resource. Apply this to queries, mutations, subscriptions, workers, exports, files, webhooks and cache keys. Knowing an order ID does not grant access.

Defense in depth

Application authorization is mandatory. PostgreSQL Row-Level Security can add protection but does not replace application authorization. If using pooled connections, tenant context must be transaction-scoped and safely reset.

Transactions and migrations

Use transactions for multi-row invariants such as order header plus order items. Test migrations against fresh and existing databases. Prefer expand-and-contract production changes: add compatible schema, deploy compatible code, backfill, and remove obsolete schema in a later release. Do not casually edit migrations already applied to shared environments.

6. Authentication, tokens and rate limits

OAuth 2.0 and OIDC

OAuth 2.0 is an authorization framework; OpenID Connect adds standardized identity/login. Use a supported flow for the selected provider. Do not invent cryptographic protocols. Provider, MFA and recovery choices remain open until approved.

Token safety

Access tokens should be short-lived. Rotate refresh tokens and support revocation/reuse detection where appropriate. Never store reusable refresh bearer tokens in plaintext. Persist protected session/token identifiers and device/session metadata. Redis can support ephemeral coordination but is not automatically the durable session source of truth.

Requested refresh lifetimes of **POS 90 days** and **web 3 days** are proposals until approved. Define idle/absolute expiry, device revocation, shared POS login and reauthentication policy.

Layered rate limiting

Combine limits by:

- IP/network;
- account identifier;
- OTP/email destination;
- authenticated user/tenant;
- GraphQL query complexity/concurrency;
- subscription connection count.

IP-only limits may block users behind shared networks. Account-only limits can be abused to lock out a victim. Combine controls with cooldowns and safe recovery.

OTP and email verification

Codes should be short-lived, one-time, rate-limited and protected at rest. Never log codes. Avoid responses that unnecessarily reveal whether an email exists. Limit both send and verify attempts and record delivery outcomes without logging secrets.

7. GraphQL queries, mutations and subscriptions

Responsibilities

- **Queries:** read dashboards, menus, orders and reports.
- **Mutations:** create orders, change kitchen status, update menu, request upgrades or approve trials.
- **Subscriptions:** deliver selected live updates such as new kitchen orders or order status changes.

Resolvers should call domain services; business rules must not be scattered across resolvers. Authorize tenant/outlet access before returning data or changing state.

Subscription security

Authenticate WebSocket connections, handle token expiry, authorize tenant/outlet at subscription creation, and enforce scope for each delivered event. Re-check membership changes where required. Do not trust client-provided topic names or IDs.

Subscriptions are not durable queues

Clients disconnect, sleep and lose network. Persist authoritative state in PostgreSQL and refetch after reconnect. Use event IDs/versions to detect gaps or duplicates where useful. Critical events need a transactional outbox or equivalent reliable publication pattern.

Commit before publish

Do not notify the kitchen about an order before its transaction commits. An outbox record written in the same transaction can be published by a worker later. Consumers must tolerate duplicate delivery.

Performance guardrails

Set pagination, query depth/complexity budgets, timeouts, rate limits and subscription connection limits. Mitigate N+1 queries using batching/DataLoader or explicit aggregates. Bound payload sizes. Use caching only when authorization and invalidation are clear.

Why REST still exists

REST may fit payment-provider webhooks, external callbacks and file workflows. Both REST and GraphQL must call the same domain services, not duplicate business rules.

8. Backend reliability, workers and concurrency

Validation

Validate payload shape, size, enums, identifiers and business constraints on the server. Frontend validation improves UX but is not a security control.

Idempotency

For retry-prone operations such as order creation, payments and billing changes, persist an idempotency key under a unique constraint. A retry must not create a duplicate action. Define behavior when a key is reused with a different payload.

Concurrency
Two kitchen users may update the same order at once. Use optimistic version checks or database locks where needed. Enforce stock/payment invariants in transactions and constraints, not only in process memory.

Worker jobs
Jobs need stable IDs, versioned payloads, timeouts, retry/backoff with jitter, idempotency, failure visibility and dead-letter/replay procedures. Durable work must not live only in memory. Retry transient failures only. If a payment provider may have accepted a request before timing out, reconcile by provider ID/idempotency key before retrying.

9. Frontend, mobile and POS
The UI presents authorized data; it is not the security boundary. Hiding a feature in the UI does not replace backend entitlement checks.

Every screen should define loading, retryable error, permission denied, empty, stale/offline and success states. Do not silently display zero when data is missing or incomplete. Use shared design tokens and accessible components.

Use platform secure storage for sensitive mobile tokens. Crash reports may include release/platform/correlation context but must exclude secrets, OTPs, card data and unnecessary personal information.

Printer, scanner, cash drawer, payment terminal and offline needs require a proof of concept. Define queued operations, idempotency, conflict handling and reconciliation before promising offline support.

10. Money, subscriptions and payment correctness

Money representation
Never use binary floating point for currency. Use integer minor units where suitable or fixed-precision decimal. Store currency with every amount and define rounding rules.

Plan changes
Requested rule: upgrades take effect immediately with proration; downgrades take effect at renewal. Failed-payment behavior, credits/refunds, add-on proration and outlet-over-limit behavior remain open.

Illustrative proration
Hypothetical 30-day period: Starter $30, Growth $60, upgrade after day 10.

- Starter usage: $30 × 10/30 = $10.
- Growth remaining-period charge: $60 × 20/30 = $40.
- Total for that period: $50.
- If $30 was already paid, additional due: $20.

This is an example, not approved pricing policy. Production must use actual timestamps, approved price versions and rounding rules.

Ledger and webhooks
Do not overwrite financial history. Use explicit credits, refunds, reversals or adjustments. Verify webhook signatures; handle duplicate/out-of-order events idempotently; reconcile provider state.

Entitlements
Compute access from plan version, subscription period, approved trial overrides, add-ons and outlet allowance. Enforce server-side in every protected operation. Cache only with reliable invalidation.

11. Observability and incident response

Track request latency/errors, DB pool and slow queries, queue depth/age, retries/dead letters, subscription connections, webhook failures and notification delivery.

Compare Sentry, GlitchTip, Firebase Crashlytics and OpenTelemetry options using current pricing, event volume, retention, coverage, privacy and self-hosting effort. No vendor is approved until recorded in the decision log.

High-severity errors should notify email and a dedicated Teams channel. Include environment, release, platform, timestamp, severity, correlation ID and sanitized diagnostics. Deduplicate alerts, retry delivery and monitor alert-delivery failure. A browser crash may prevent its own report, so add external availability checks. Mobile crash reports may arrive later. Never log passwords, OTPs, tokens, card data or unnecessary personal data.

12. Docker, Terraform, EC2 and ECS

Container rules

Build immutable images; run as non-root; keep secrets/config outside the image; implement health/readiness checks and graceful shutdown; store no durable state on container-local disk.

API and worker

Use shared domain logic where practical but separate API/worker entrypoints and scaling. Workers must not depend on one API instance's memory.

EC2-to-ECS portability

The same image and business logic should run on EC2 Docker and ECS without a code rewrite. Infrastructure configuration, task definitions, networking, IAM, secrets and scaling settings will change. Prove portability in a POC.

Terraform

Protect remote state, use least-privilege IAM, separate environments, tag resources, avoid secrets in source/state, review plans and protect production resources from accidental destruction.

Load tests

Test realistic order peaks, GraphQL queries, subscriptions, workers and reports. Record environment, workload, p95/p99 latency, errors, CPU/memory, DB saturation, queue age and projected cost. Optimize measured bottlenecks rather than adding microservices by assumption.

13. Security checklist

Before coding, identify data sensitivity, tenant/outlet scope, roles, abuse cases, providers, audit needs and state transitions.

Before merge, test:

- allowed and denied roles;
- cross-tenant and cross-outlet access;
- guessed IDs;
- expired/revoked sessions;
- duplicate requests and concurrent updates;
- malformed input;
- provider timeout/retry and webhook replay;
- secret redaction;
- migration compatibility and rollback/forward-fix plan.

14. Testing strategy

- **Unit:** order totals, kitchen transitions, entitlement evaluation, proration and pure business rules.
- **Integration:** PostgreSQL transactions, constraints, migrations, repositories, outbox and worker idempotency. Mocks alone cannot prove DB isolation or locking.
- **End-to-end:** login, outlet access, order creation, kitchen update, trial request, upgrade, webhook and dashboard metrics.
- **Realtime:** subscription authorization, duplicate/out-of-order events, reconnect, gap resync and multi-instance delivery.
- **Load/recovery:** agreed workload, backup restore in a clean environment, tenant export, alert delivery and failure handling.

Report exact commands and actual results. If a test was not run, say so. Never claim scale, security or recovery is proven without evidence.

15. Local setup and developer workflow

1. Use repository-pinned Node/package-manager versions, Docker, Git and required mobile SDKs.

2. Follow the lockfile and documented setup; do not upgrade major dependencies during unrelated work.

3. Use an ignored local `.env`; commit only placeholder names in `.env.example`.

4. Create a branch named with the task ID.

5. Write tests, implement, run lint/typecheck/tests/build, inspect diff and secret scan.

6. Open a focused PR with acceptance evidence.

7. Review migration SQL on fresh and existing databases; document rollback or forward-fix.

8. Update docs when behavior changes.

Stop and ask if requirements conflict, a provider is undecided, tenant boundaries are unclear, payment behavior is ambiguous or scope would expand.

16. Open decisions for project meetings

- Plan feature matrix/prices; Business trial duration/expiry; add-on and extra outlet pricing; downgrade when outlet count exceeds allowance; failed-payment grace; cancellation/refund/tax policy.
- Payer/tenant/brand/franchise model; independent franchisee boundaries; consolidated reporting; roles; retention/export/deletion.
- Identity provider, OIDC/MFA, refresh lifetimes, idle/absolute expiry, shared POS device policy and OTP provider.
- Cloud region, EC2 Docker versus ECS starting point, queue/subscription transport, Redis need, backup retention, availability/RPO/RTO and 10,000+ tenant workload assumptions.
- SaaS billing provider, diner payment gateway/methods, monitoring provider, alert severity/escalation, email/SMS provider and fees.
- POS hardware compatibility, offline requirements and sync conflict policy, mobile/web feature parity.

17. Reusable junior developer task template

Task ID:

Role: Senior [specialist] engineer

Objective:

Required documents:

Allowed modules/files:

Out of scope:

Dependencies:

Implementation steps:

Acceptance criteria:

Required tests:

Tenant/security checks:

Migration/deployment impact:

PR evidence:

Open questions (ask; do not guess):

Example: tenant authorization task

Objective: users can access only resources in their authorized tenant/outlet. Inspect auth context; resolve membership server-side; implement authorization service; apply it to one assigned module; add positive and negative integration tests; verify logs redact tokens; document the module contract.

Acceptance: authorized requests succeed; cross-tenant/outlet and guessed-ID attempts fail; CI runs the tests. Do not retrofit every module in one oversized PR.

Appendix — Pre-merge checklist

- ☐ Acceptance criteria pass and evidence is attached.
- ☐ Tenant/outlet authorization is enforced server-side and negative tests pass.
- ☐ Input validation, idempotency and concurrency were considered.
- ☐ No secrets or sensitive data are logged or committed.
- ☐ Relevant unit, integration and E2E tests were run.
- ☐ Migrations and deployment/rollback notes are reviewed.
- ☐ Monitoring and retry/failure behavior are covered where relevant.
- ☐ Knowledge base and decision log are updated when behavior changes.